SYSTEMS ANALYSIS AND DESIGN

Classes & Objects in Python

Part 1 · Thinking in Objects

Dr. Pejman Ebrahimi

Today’s roadmap

Four steps from loose variables to your own classes



01

Why objects?

Grouping data that belongs together



02

Your first class

class, __init__, self and attributes



03

Methods

Giving objects behaviour



04

Many objects & UML

Lists of objects, functions, class diagrams

By the end of today you can…



Explain the difference between a **class** and an **object**



Write a class with a constructor `__init__` and attributes



Add **methods** that read or change an object's data



Store many objects in a **list** and loop over them



Translate a simple **UML class diagram** into Python code

01

Why objects?

From loose variables to well-organised data

The problem: data that belongs together

Imagine storing information about the students in this room

loose_variables.py

```
# Student 1
name1 = "Anna"
age1 = 21
major1 = "Business Informatics"

# Student 2
name2 = "Ben"
age2 = 23
major2 = "Economics"

# ... and 60 more students?
```



What goes wrong?

- Hundreds of variables to keep track of
- Nothing tells Python that `name1` and `age1` belong to the **same** person
- Easy to mix up `age1` with `age2`
- No place to put actions like “celebrate a birthday”

A first fix: group the data

Tuples and dictionaries help — but only half way

with a tuple

```
anna = ("Anna", 21, "Business")
print(anna[1])   # index 1 = ???
```

with a dictionary

```
anna = {"name": "Anna", "age": 21}
print(anna["age"])
print(anna["agee"])  # KeyError!
```

Tuple

Groups data, but positions like `[1]` have no meaning. Cannot be changed later.

Dictionary

Readable keys, but no fixed structure and **no behaviour**: data and actions stay separate.

Objects bundle data and behaviour

An object knows things and can do things



Attributes

What the object **knows**

Data stored inside it:
colour, brand, speed



my_car

```
color = "red"
speed = 0
```

```
accelerate()
brake()
```



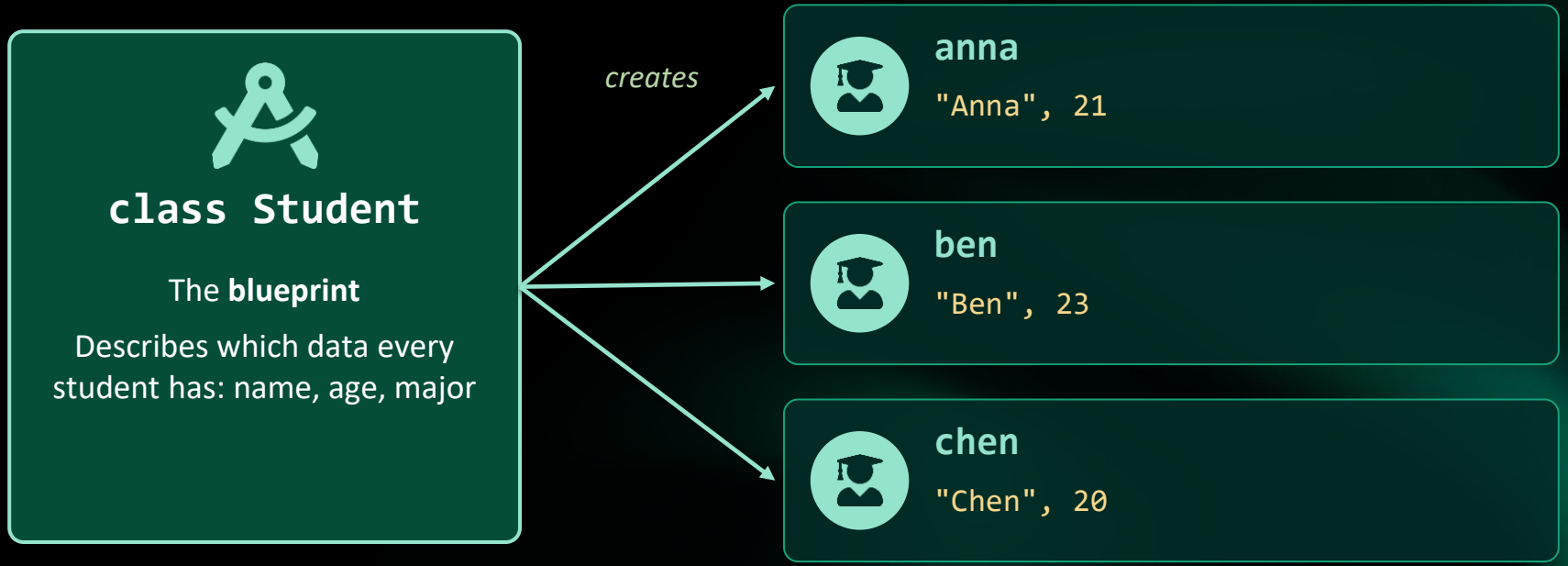
Methods

What the object **can do**

Actions that use or change its data: accelerate, brake

Class vs object: the blueprint and the real thing

One class → as many objects as you need



Each object is called an **instance** of the class. Same structure, different values.

Surprise: you already use objects every day

In Python, every value is an object of some class

objects_everywhere.py

```
name = "anna"
print(type(name))
print(name.upper())      # str method

numbers = [3, 1, 2]
print(type(numbers))
numbers.append(4)        # list method
print(numbers)
```

Output

```
<class 'str'>
ANNA
<class 'list'>
[3, 1, 2, 4]
```

`str` and `list` are **classes**.

The dot in `name.upper()` means: “ask this object to do something”.

Today we learn to build **our own** classes.

The vocabulary of OOP

Six words you will hear in every lecture from now on



Class

A blueprint that defines the structure and behaviour of objects



Object / instance

One concrete “thing” created from a class



Attribute

A variable that belongs to an object, e.g. `anna.age`



Method

A function that belongs to an object, e.g. `acc.deposit(50)`



Constructor

The special method `__init__` that sets up a new object



self

Inside the class: “the object I am working on right now”

02

Your first class

class, __init__, self and attributes

Defining a class

The keyword class, a name in CamelCase, and a body

first_class.py

```
class Student:
    pass          # empty body for now

anna = Student()  # create an object
ben = Student()   # another object

print(type(anna))
print(anna is ben)
```

Output

```
<class '__main__.Student'>
False
```

Rules of thumb

- Class names use **CamelCase**: `Student`, `BankAccount`, `LibraryBook`
- Variables and methods use **snake_case**: `anna`, `get_balance`
- Call the class like a function, `Student()`, to create an object
- `pass` means “nothing here yet”

The constructor: __init__

It runs automatically every time a new object is created

student.py

```
class Student:
    def __init__(self, name, age):    # 1, 2
        self.name = name               # 3
        self.age = age

anna = Student("Anna", 21)             # 4
ben = Student("Ben", 23)
print(anna.name, anna.age)
print(ben.name, ben.age)
```

Output

```
Anna 21
Ben 23
```

1. Two underscores on each side: `__init__`
2. `self` is the new object being built
3. `self.name` (attribute) gets the value of `name` (parameter)
4. You pass only `name` and `age`; Python passes `self` automatically

What happens when you write Student("Anna", 21)?

Four steps, every single time

1 — Create

Python creates a new, empty Student object

→

2 — Call __init__

It runs `__init__` with `self` = the new object

→

3 — Fill

`self.name = "Anna"` stores the data inside that object

→

4 — Return

The finished object is stored in the variable `anna`

```
anna = Student("Anna", 21)     # steps 1 → 4 happen in this one line
```

Understanding self

self is simply the object that the method is working on

self_demo.py

```python
class Student:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hi, I am", self.name)

anna = Student("Anna")
ben = Student("Ben")
anna.greet()   # here self is anna
ben.greet()    # here self is ben
```



Think of it like this

`anna.greet()` is really

`Student.greet(anna)`

Python fills in `self` for you with the object before the dot.

Output

```
Hi, I am Anna
Hi, I am Ben
```

Each object has its own data

Changing one object does not change the others

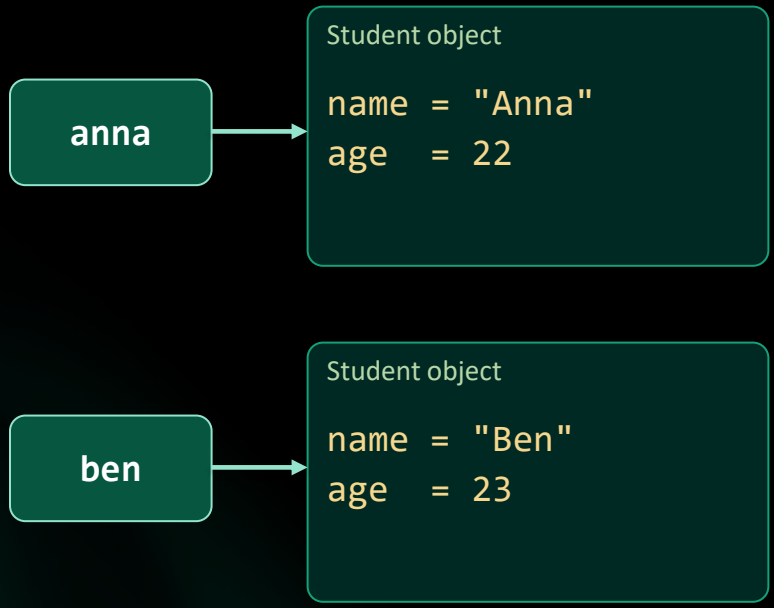
independent.py

```
anna = Student("Anna", 21)
ben = Student("Ben", 23)

anna.age = 22       # birthday!
print(anna.age)
print(ben.age)      # unchanged
```

Output

```
22
23
```



variables point to separate objects in memory

Attributes do not all have to be parameters

Use default values and start values set inside __init__

defaults.py

```
class Student:
    def __init__(self, name, age, country="LI"):
        self.name = name
        self.age = age
        self.country = country   # default value
        self.courses = []        # starts empty

anna = Student("Anna", 21)
ben = Student("Ben", 23, "AT")
anna.courses.append("SAD")
print(anna.country, anna.courses)
print(ben.country, ben.courses)
```

Output

```
LI ['SAD']
AT []
```



Tip

Give every attribute a value in `__init__`, even if it is just `0`, `""` or `[]`. Then all objects have the same shape.

In-class task 1: your first class

10 min

Work in pairs · 10 minutes



Your task

- Write a class `Book` with the attributes `title`, `author` and `pages`
- Add an attribute `is_read` that always starts as `False`
- Create two different books
- Mark one book as read and print both books' attributes

starter code

```
class Book:
    def __init__(self, ...):
        ...

b1 = Book("Dune", "Herbert", 412)
```

Expected output (example)

```
Dune Herbert 412 True
Emma Austen 474 False
```

03

Methods

Giving objects behaviour

Methods: functions that belong to an object

Defined inside the class, always with self as the first parameter

bank_account.py

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def show(self):
        print(self.owner, "has", self.balance)

acc = BankAccount("Anna", 100)
acc.deposit(50)
acc.show()
```

Output

```
Anna has 150
```



How to call a method

`object.method(arguments)`

`acc.deposit(50)` → inside the method, `self` is `acc` and `amount` is `50`

The method can read and change `self.balance`

Two kinds of methods

Some change the object, some only give back information

1. Change the object's state

```
def deposit(self, amount):
    self.balance += amount   # changes

def withdraw(self, amount):
    self.balance -= amount   # changes
```

2. Return information

```
def get_balance(self):
    return self.balance

def is_rich(self):
    return self.balance > 1_000_000
```



Commands

Usually return nothing. You call them for their effect: `acc.deposit(50)`



Queries

Use `return`. You use the answer: `print(acc.get_balance())`

Methods can protect the rules of your object

The account decides itself whether a withdrawal is allowed

withdraw.py

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            print("Not enough money!")
            return False
        self.balance -= amount
        return True

acc = BankAccount("Anna", 100)
print(acc.withdraw(30), acc.balance)
print(acc.withdraw(500), acc.balance)
```

Output

```
True 70
Not enough money!
False 70
```



Why is this good?

The rule “no negative balance” lives in **one place**.

Every part of the program that uses the account gets the same, correct behaviour.

Methods can call other methods

Use self.method_name() inside the class

rectangle.py

```python
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def describe(self):
        print("Area:", self.area())
        print("Perimeter:", self.perimeter())

Rectangle(4, 3).describe()
```

Output

```
Area: 12
Perimeter: 14
```



Remember

Inside the class you always need `self.` to reach attributes **and** other methods.

Printing objects nicely with __str__

Tell Python what your object should look like as text

before

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

anna = Student("Anna", 21)
print(anna)
```

Output

```
<__main__.Student object at 0x7f59dae64ad0>
```

after

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} ({self.age})"

anna = Student("Anna", 21)
print(anna)
```

Output

```
Anna (21)
```

Four classic beginner mistakes

Read the error message: it usually tells you exactly what is wrong

Forgot self in the definition

```
def greet():
```

```
TypeError: Student.greet() takes 0 positional
arguments but 1 was given
```

Forgot self. when storing

```
name = name   # lost!
```

```
AttributeError: 'Student' object has no
attribute 'name'
```

Forgot () when calling

```
print(anna.greet)
```

```
<bound method Student.greet of ...>
```

Wrong number of arguments

```
Student("Anna")
```

```
TypeError: Student.__init__() missing 1 required
positional argument: 'age'
```

In-class task 2: a Counter class

12 min

Work in pairs · 12 minutes



Write a class Counter

- Attribute `value` starts at `0`
- `increase()` adds 1
- `decrease()` subtracts 1, but never goes below 0
- `reset()` sets the value back to 0
- `__str__` returns e.g. `Counter: 2`
- **Bonus:** `increase(step=1)` with an optional step

test it like this

```
c = Counter()
c.increase()
c.increase()
c.increase()
c.decrease()
print(c)
```

Expected output

```
Counter: 2
```

04

Many objects & UML

Lists of objects, functions and class diagrams

University of Liechtenstein

Dr. Pejman Ebrahimi

A list of objects

Objects can be stored in lists like any other value

list_of_objects.py

```
students = [Student("Anna", 21),
            Student("Ben", 23),
            Student("Chen", 20)]

for s in students:
    print(s.name, "is", s.age)

total = 0
for s in students:
    total += s.age
average = total / len(students)
print("Average age:", round(average, 1))
```

Output

```
Anna is 21
Ben is 23
Chen is 20
Average age: 21.3
```



Pattern

Loop over the list; each item `s` **is** a Student, so `s.name` and `s.age` work.

Objects as function arguments and return values

Functions can receive objects and give objects back

functions.py

```
def oldest(students):
    best = students[0]
    for s in students:
        if s.age > best.age:
            best = s
    return best          # a Student object

group = [Student("Anna", 21), Student("Ben", 23),
         Student("Chen", 20)]
winner = oldest(group)
print(winner.name, winner.age)
```

Output

```
Ben 23
```



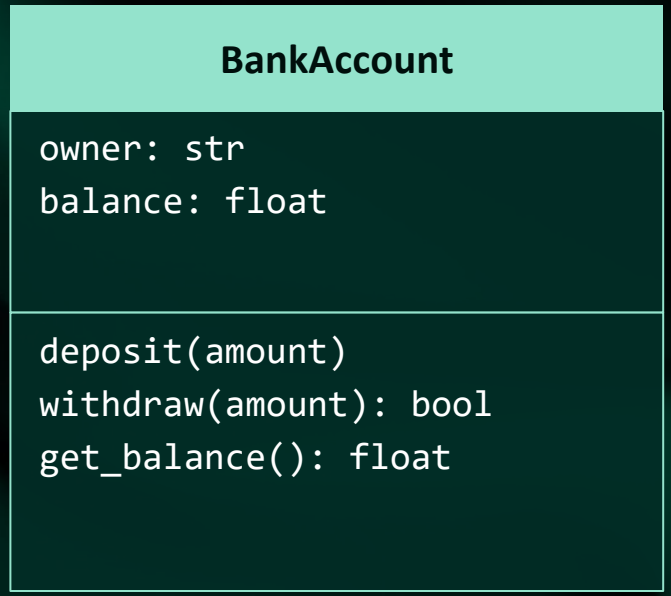
Important

The function returns the **same object** that is in the list, not a copy.

Changing `winner.age` would also change it inside `group`.

From UML class diagram to Python

In Systems Analysis & Design we model first, then code



1 name

2 attributes

3 methods

Three compartments: **name**, **attributes**, **methods**

bank_account.py

```python
class BankAccount:                  # 1

    def __init__(self, owner, balance):
        self.owner = owner          # 2
        self.balance = balance      # 2

    def deposit(self, amount):      # 3
        ...
    def withdraw(self, amount):     # 3
        ...
    def get_balance(self):          # 3
        ...
```

In-class task 3: model, then code

15 min

Work in groups of three · 15 minutes

Step 1 · Model (on paper)

Draw a UML class diagram for a university `Course`:

- name and credits (ECTS)
- a list of enrolled student names
- `enroll(name)` adds a student
- `number_of_students()` returns the count
- `is_full()` returns `True` when 30 or more are enrolled

Step 2 · Code (in Python)

- Implement the class from your diagram
- Add a `__str__` like `SAD (6 ECTS): 2 students`
- Create a course, enrol three students and print it

Compare your diagram with your neighbours': did you all choose the same names and types?

Summary: Part 1 on one slide

cheat_sheet.py

```python
class Student:                          # blueprint
    def __init__(self, name, age):      # constructor
        self.name = name                # attributes
        self.age = age

    def birthday(self):                 # method (command)
        self.age += 1

    def is_adult(self):                 # method (query)
        return self.age >= 18

    def __str__(self):                  # nice printing
        return f"{self.name} ({self.age})"

anna = Student("Anna", 21)              # object
anna.birthday()
print(anna, anna.is_adult())
```

Key takeaways

- A **class** describes; an **object** is a real instance
- `__init__` sets up the attributes
- `self` = the current object
- Methods = behaviour; call them with a dot
- UML class diagram ↔ class code

Homework 1: Song & Playlist

Submit one .py file on Moodle before the next lecture

Requirements

- `Song` with `title`, `artist`, `seconds`
- `Song.__str__` → `Artist – Title (3:25)`
- `Playlist` with a `name` and a list of songs (starts empty)
- `add_song(song)`
- `total_duration()` returns total seconds
- `longest_song()` returns the Song object, or `None` if the playlist is empty
- `print_all()` prints every song, numbered

example use

```
p = Playlist("Study mix")
p.add_song(Song("Clocks", "Coldplay", 307))
p.add_song(Song("Holocene", "Bon Iver", 337))
p.print_all()
print(p.total_duration(), p.longest_song())
```

Expected output

```
1. Coldplay – Clocks (5:07)
2. Bon Iver – Holocene (5:37)
644 Bon Iver – Holocene (5:37)
```

Thank you!

Dr. Pejman Ebrahimi

Do you have any questions?

 pejman.ebrahimi@uni.li